VULNERABILITY TESTING / LAB REPORT

UPLOAD VULNERABILITIES

Web Application Penetration Testing – Internship Report

Organization: IqraSec Academy
Platform: TryHackMe

COVER PAGE

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT)
Topic	File Upload Vulnerabilities
Platform	TryHackMe
Number of Labs	1
Completion Date	25-10-2026

This report documents the objective, vulnerability concept, testing methodology, evidence, impact, remediation, and learning outcomes of the Upload Vulnerabilities lab.

Lab 1: Upload Vulnerabilities

1. Lab Name

Upload Vulnerabilities

Platform: TryHackMe
Room URL: https://tryhackme.com/room/uploadvulns

2. Objective

The objective of this lab is to understand file upload vulnerabilities in web applications and learn how insecure upload functionality can expose a server to unauthorized file access or execution.

The main objectives are:

- Understand common file upload security risks.

- Identify how applications validate uploaded files.
- Understand file extension and MIME-type validation.
- Learn about insecure file storage and access controls.
- Examine the risks of uploading executable or unexpected file types.
- Understand how to test upload functionality safely.
- Learn secure file upload implementation and remediation techniques.

3. Vulnerability / Concept

What are File Upload Vulnerabilities?

File upload vulnerabilities occur when a web application fails to validate, store, or serve uploaded files securely. Improper controls can allow users to upload unexpected content, overwrite files, expose sensitive information, or potentially execute malicious code on the server.

The severity depends on the application's upload logic, server configuration, file storage location, and execution permissions.

Common Types of File Upload Issues

1. Insufficient File Extension Validation

An application may accept file extensions that should not be permitted. Extension checks alone are not a reliable security boundary.

2. Insecure MIME-Type Validation

Some applications rely only on the client-provided Content-Type header. This value can be manipulated and should not be trusted as the sole validation mechanism.

3. Weak File Content Validation

An application may accept files without verifying whether their contents match the expected file format.

4. Insecure File Storage

Uploaded files may be stored in publicly accessible directories or locations where they can be accessed without appropriate authorization.

5. Unsafe Server-Side File Execution

If uploaded files are stored in a location where the server interprets them as executable code, a successful upload may lead to server-side code execution.

6. Insufficient File Size and Resource Controls

Missing limits can allow oversized or excessive uploads that consume server resources.

Common Indicators

- Unexpected file types are accepted.
- Validation depends only on the file extension or client-supplied MIME type.
- Uploaded files are stored in publicly accessible locations.

- File names are not safely generated or normalized.
- Uploaded content can be interpreted as executable code.
- File size, authorization, or storage restrictions are missing.

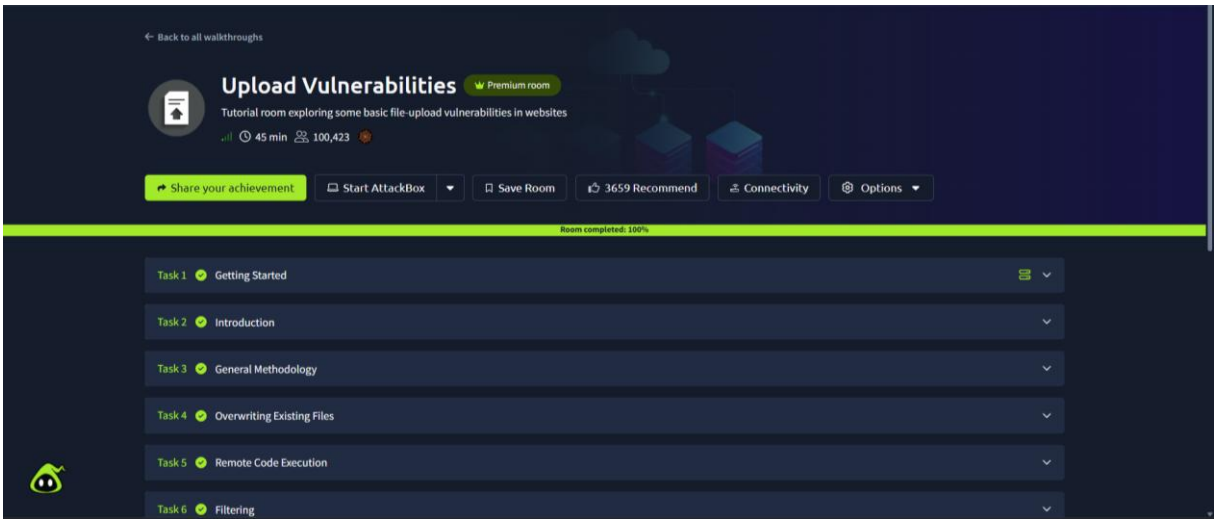
4. Testing Approach

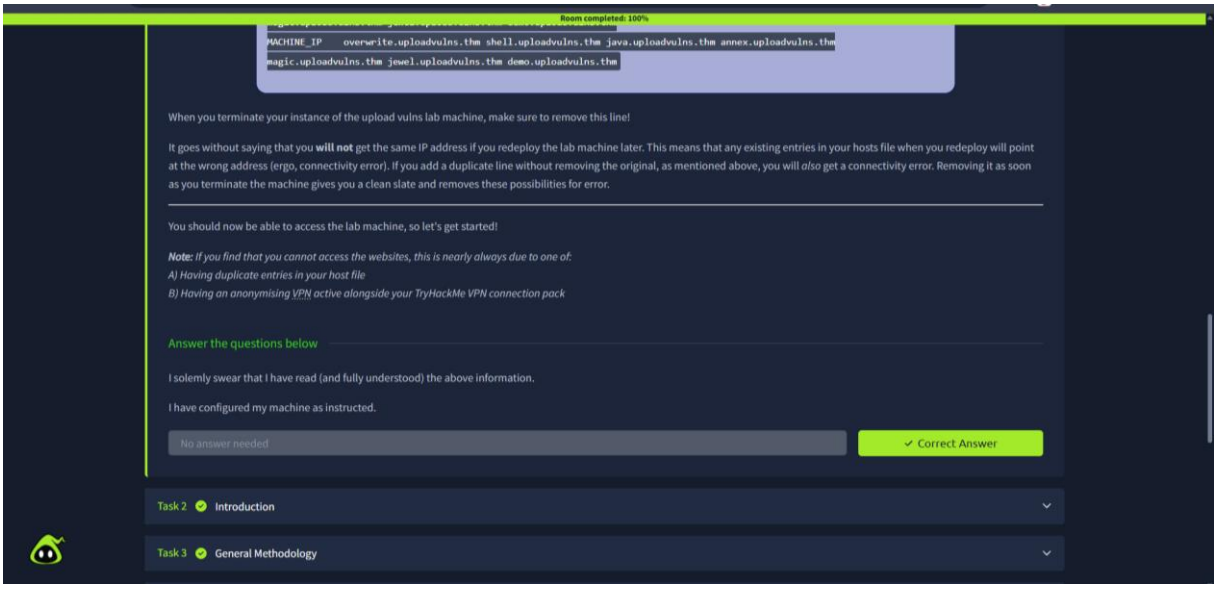
The following methodology can be used in the authorized TryHackMe lab environment.

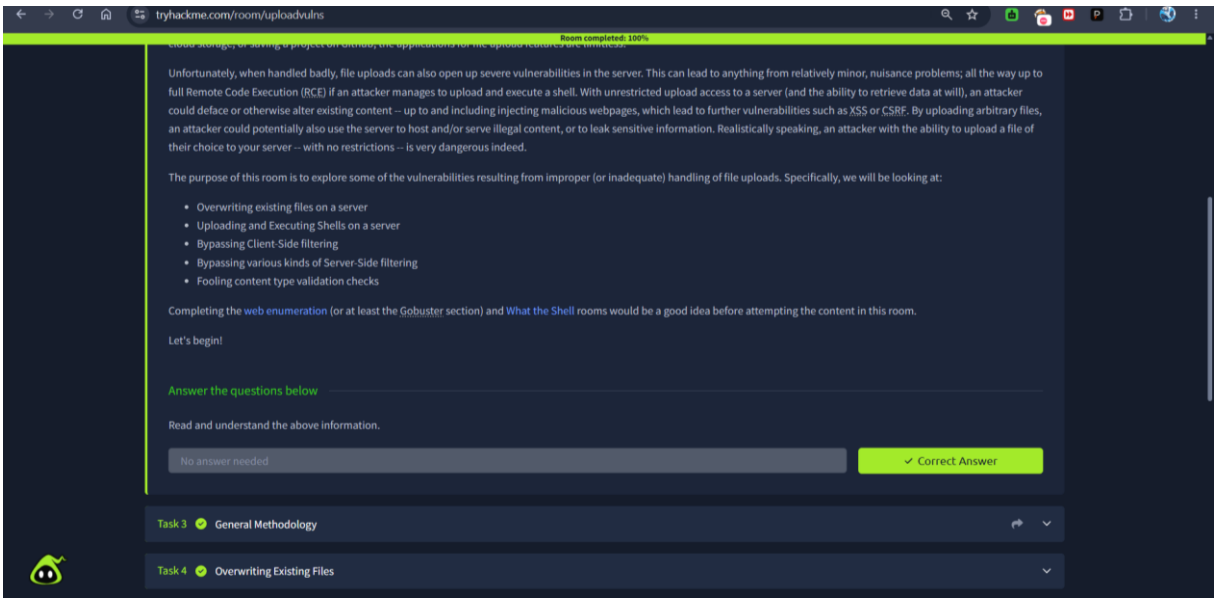
1. Open the Upload Vulnerabilities room on TryHackMe.
2. Read the room instructions and understand the assigned tasks.
3. Start the target machine or lab environment if required.
4. Open the target web application in the provided browser.
5. Locate the file upload functionality.
6. Identify the types of files the application is intended to accept.
7. Upload a normal permitted file to establish baseline behavior.
8. Observe the server response and any validation messages.
9. Review how the application handles file extensions.
10. Examine whether MIME-type validation is discussed or demonstrated in the lab.
11. Follow the lab instructions to test its designated validation controls.
12. Observe whether the application rejects or accepts the test file.
13. Review how the application handles file names and duplicate uploads.
14. Examine the upload response for information about the saved file or its location.
15. Determine whether uploaded files can be accessed through the application's intended retrieval mechanism.
16. Review whether uploaded files are stored outside executable web directories, where applicable.
17. Follow the lab's instructions to examine any additional upload-related security weaknesses.
18. Record the actual requests, responses, and outcomes.
19. Capture screenshots that demonstrate the test and its result.
20. Document confirmed findings, potential impact, and recommended remediation.

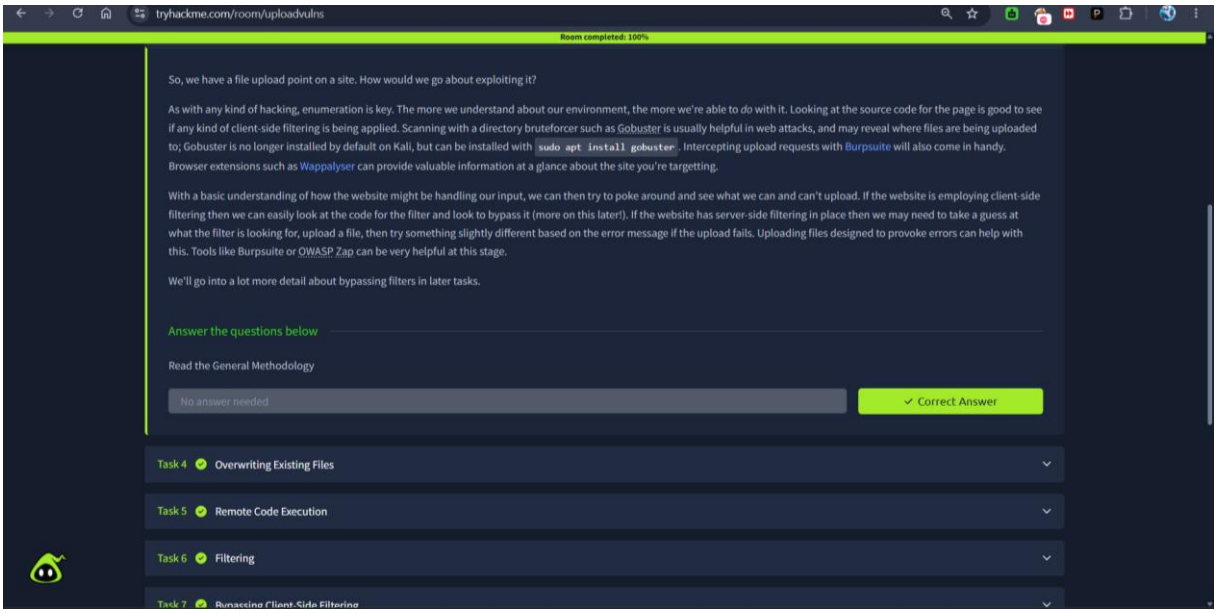
Testing note: Perform tests only against the assigned lab environment. Do not claim that a file upload vulnerability was successfully exploited unless the observed result supports that conclusion.

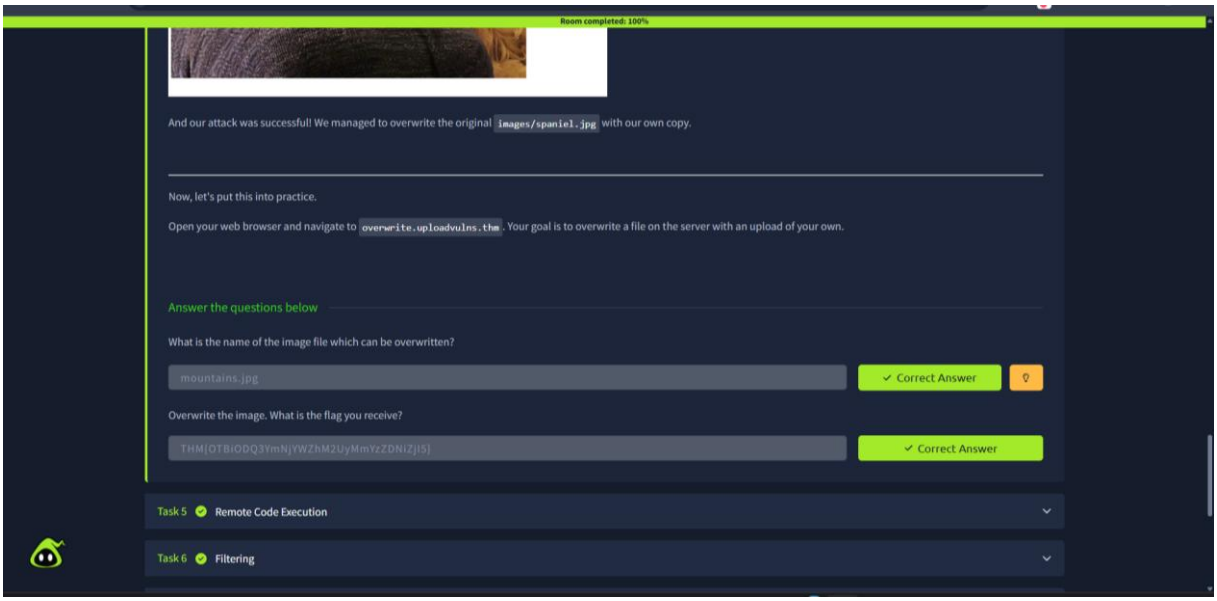
5. Evidence / Screenshots

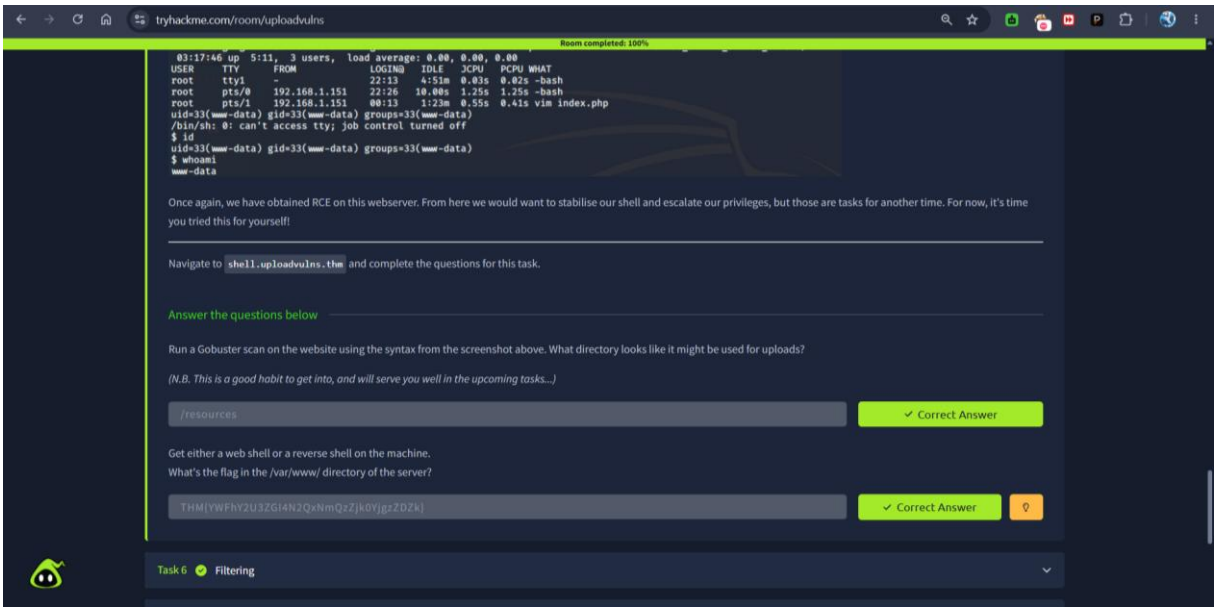

Back to all walkthroughs
Upload Vulnerabilities
Premium room
Tutorial room exploring some basic file-upload vulnerabilities in websites
45 min
100,423
Share your achievement
Start AttackBox
Save Room
3659 Recommend
Connectivity
Options
Room completed: 100%
Task 1 Getting Started
Task 2 Introduction
Task 3 General Methodology
Task 4 Overwriting Existing Files
Task 5 Remote Code Execution
Task 6 Filtering

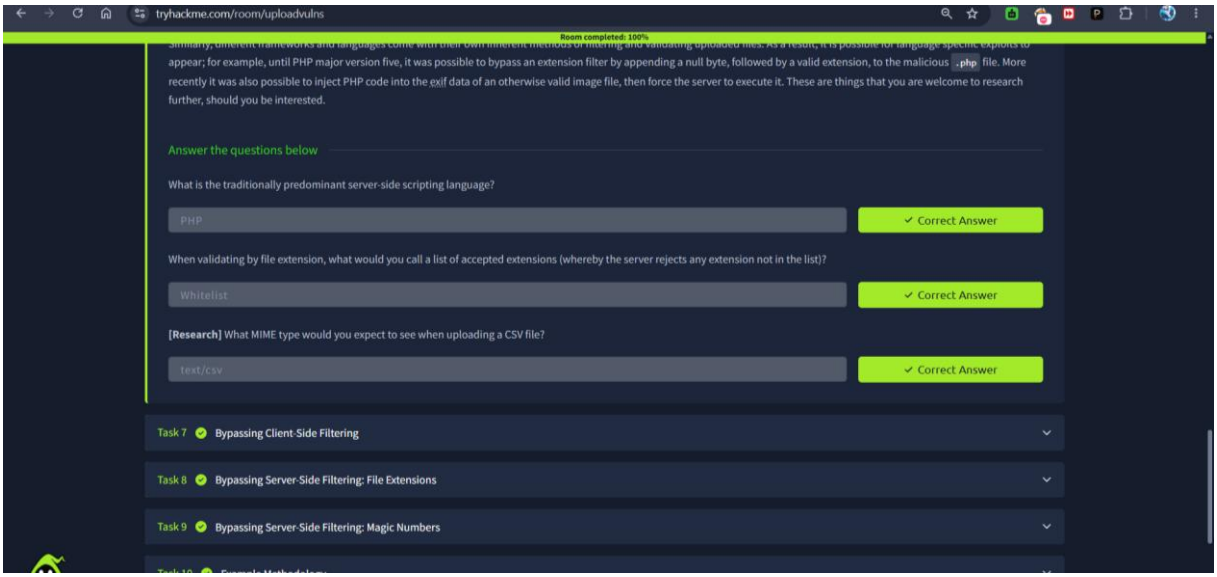

Room completed: 100%
MACHINE_IP overwrite.uploadvulns.thm shell.uploadvulns.thm java.uploadvulns.thm annex.uploadvulns.thm
magic.uploadvulns.thm jewel.uploadvulns.thm demo.uploadvulns.thm
When you terminate your instance of the upload vulns lab machine, make sure to remove this line!
It goes without saying that you will not get the same IP address if you redeploy the lab machine later. This means that any existing entries in your hosts file when you redeploy will point at the wrong address (ergo, connectivity error). If you add a duplicate line without removing the original, as mentioned above, you will also get a connectivity error. Removing it as soon as you terminate the machine gives you a clean slate and removes these possibilities for error.
You should now be able to access the lab machine, so let's get started!
Note: If you find that you cannot access the websites, this is nearly always due to one of:
A) Having duplicate entries in your host file
B) Having an anonymising VPN active alongside your TryHackMe VPN connection pack
Answer the questions below
I solemly swear that I have read (and fully understood) the above information.
I have configured my machine as instructed.
No answer needed
✓ Correct Answer
Task 2 Introduction
Task 3 General Methodology

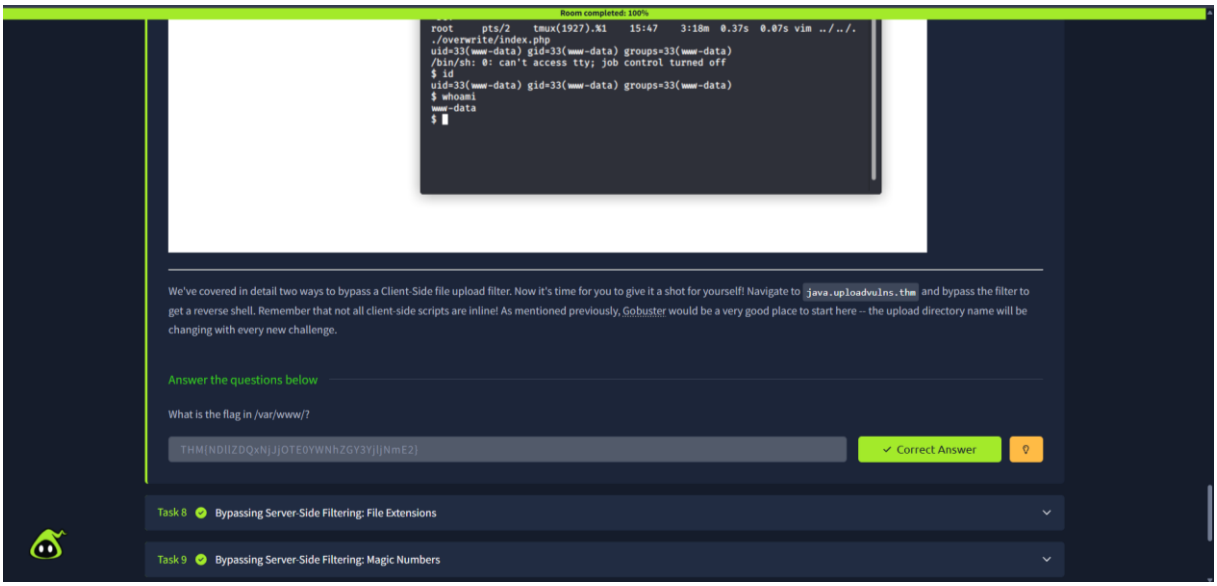

tryhackme.com/room/uploadvulns
Room completed: 100%
Unfortunately, when handled badly, file uploads can also open up severe vulnerabilities in the server. This can lead to anything from relatively minor, nuisance problems; all the way up to full Remote Code Execution (RCE) if an attacker manages to upload and execute a shell. With unrestricted upload access to a server (and the ability to retrieve data at will), an attacker could deface or otherwise alter existing content -- up to and including injecting malicious webpages, which lead to further vulnerabilities such as XSS or CSRF. By uploading arbitrary files, an attacker could potentially also use the server to host and/or serve illegal content, or to leak sensitive information. Realistically speaking, an attacker with the ability to upload a file of their choice to your server -- with no restrictions -- is very dangerous indeed.
The purpose of this room is to explore some of the vulnerabilities resulting from improper (or inadequate) handling of file uploads. Specifically, we will be looking at:
Overwriting existing files on a server
Uploading and Executing Shells on a server
Bypassing Client-Side filtering
Bypassing various kinds of Server-Side filtering
Fooling content type validation checks
Completing the web enumeration (or at least the Gobuster section) and What the Shell rooms would be a good idea before attempting the content in this room.
Let's begin!
Answer the questions below
Read and understand the above information.
No answer needed
✓ Correct Answer
Task 3 General Methodology
Task 4 Overwriting Existing Files

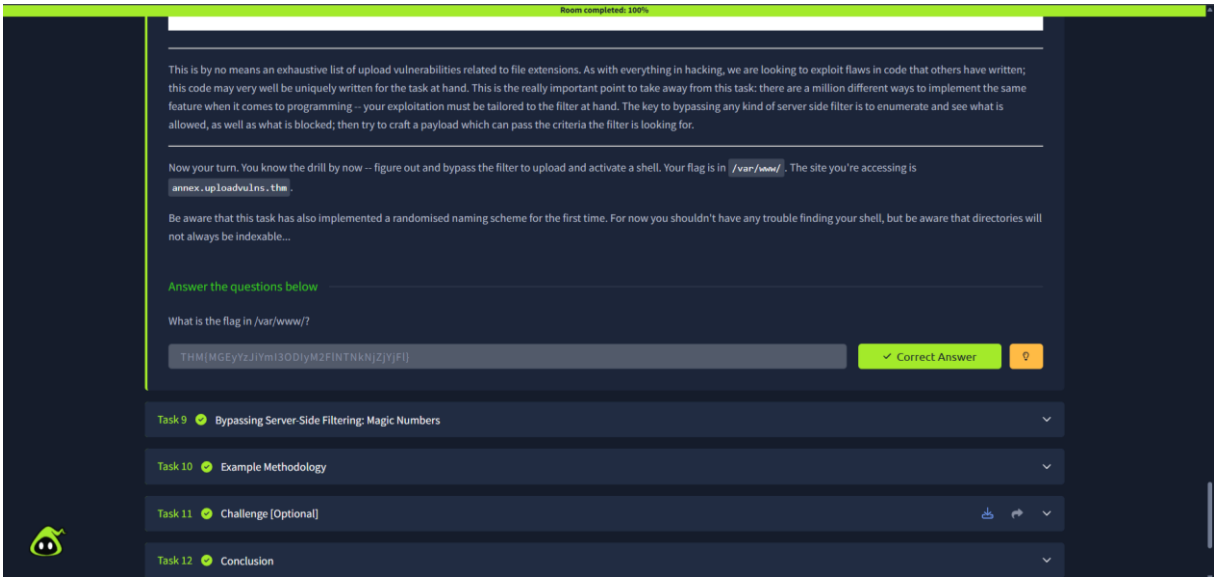

tryhackme.com/room/uploadvulns
Room completed: 100%
So, we have a file upload point on a site. How would we go about exploiting it?
As with any kind of hacking, enumeration is key. The more we understand about our environment, the more we're able to do with it. Looking at the source code for the page is good to see if any kind of client-side filtering is being applied. Scanning with a directory bruteforcer such as Gobuster is usually helpful in web attacks, and may reveal where files are being uploaded to; Gobuster is no longer installed by default on Kali, but can be installed with sudo apt install gobuster. Intercepting upload requests with Burpsuite will also come in handy. Browser extensions such as Wappalyser can provide valuable information at a glance about the site you're targetting.
With a basic understanding of how the website might be handling our input, we can then try to poke around and see what we can and can't upload. If the website is employing client-side filtering then we can easily look at the code for the filter and look to bypass it (more on this later!). If the website has server-side filtering in place then we may need to take a guess at what the filter is looking for, upload a file, then try something slightly different based on the error message if the upload fails. Uploading files designed to provoke errors can help with this. Tools like Burpsuite or OWASP Zap can be very helpful at this stage.
We'll go into a lot more detail about bypassing filters in later tasks.
Answer the questions below
Read the General Methodology
No answer needed
✓ Correct Answer
Task 4 Overwriting Existing Files
Task 5 Remote Code Execution
Task 6 Filtering
Task 7 Bypassing Client-Side Filtering

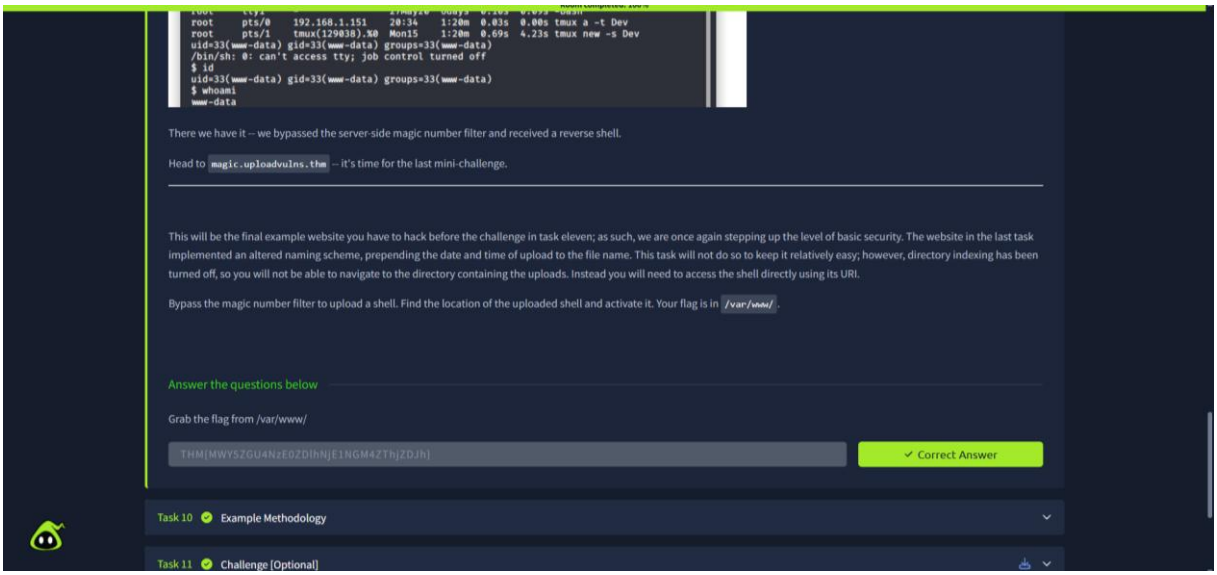

Room completed: 100%
And our attack was successful! We managed to overwrite the original images/spaniel.jpg with our own copy.
Now, let's put this into practice.
Open your web browser and navigate to overwrite.uploadvulns.thm. Your goal is to overwrite a file on the server with an upload of your own.
Answer the questions below
What is the name of the image file which can be overwritten?
mountains.jpg
✓ Correct Answer
Overwrite the image. What is the flag you receive?
THM{OTBiODQ3YmNjYWZhM2UyMmYzZDNiZjI5}
✓ Correct Answer
Task 5 Remote Code Execution
Task 6 Filtering

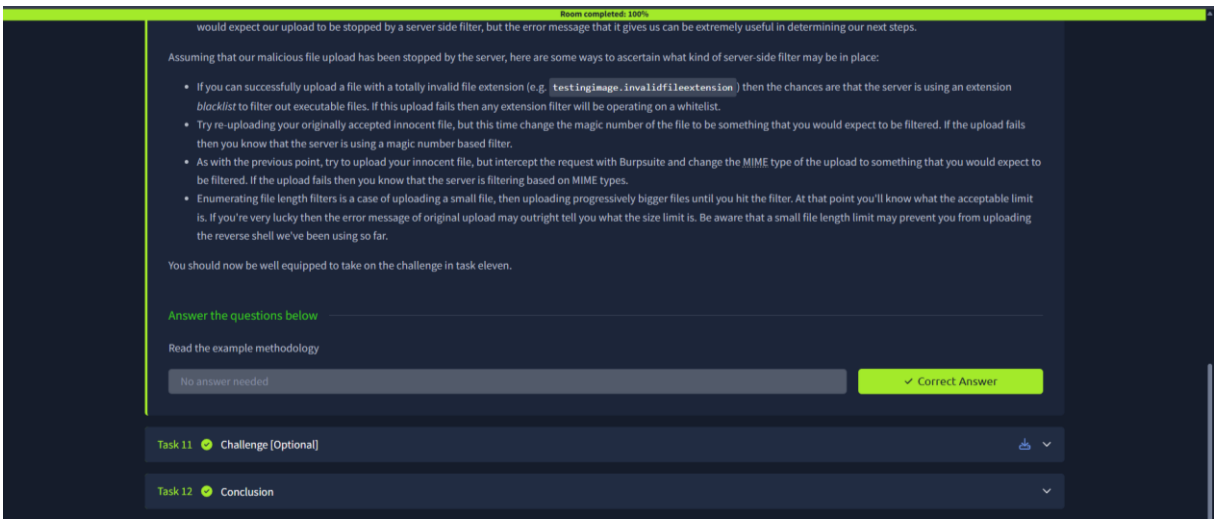

tryhackme.com/room/uploadvulns
Room completed: 100%
03:17:46 up 5:11, 3 users, load average: 0.00, 0.00, 0.00
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
root tty1 - 22:13 4:51m 0.03s 0.02s -bash
root pts/0 192.168.1.151 22:26 10.00s 1.25s 1.25s -bash
root pts/1 192.168.1.151 00:13 1:23m 0.55s 0.41s vim index.php
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ whoami
www-data
Once again, we have obtained RCE on this webserver. From here we would want to stabilise our shell and escalate our privileges, but those are tasks for another time. For now, it's time you tried this for yourself!
Navigate to shell.uploadvulns.thm and complete the questions for this task.
Answer the questions below
Run a Gobuster scan on the website using the syntax from the screenshot above. What directory looks like it might be used for uploads?
(N.B. This is a good habit to get into, and will serve you well in the upcoming tasks...)
/resources
✓ Correct Answer
Get either a web shell or a reverse shell on the machine.
What's the flag in the /var/www/ directory of the server?
THM{YWFhY2U3ZGI4N2QxNmQzZjk0YjgzZDZk}
✓ Correct Answer
Task 6 Filtering

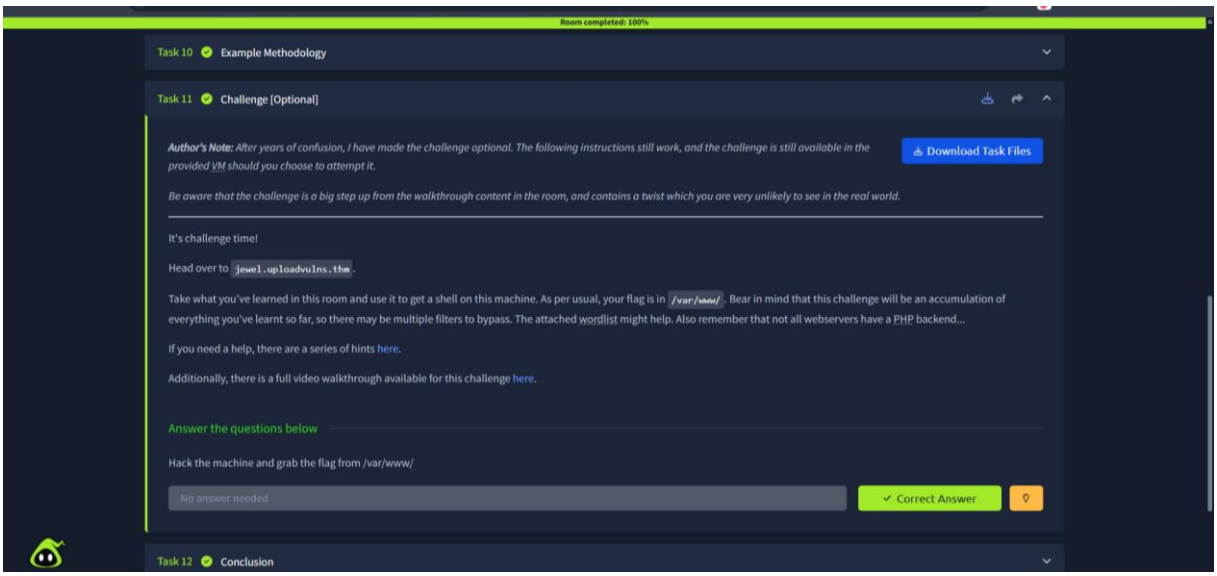

tryhackme.com/room/uploadvulns
Room completed: 100%
appear; for example, until PHP major version five, it was possible to bypass an extension filter by appending a null byte, followed by a valid extension, to the malicious .php file. More recently it was also possible to inject PHP code into the exif data of an otherwise valid image file, then force the server to execute it. These are things that you are welcome to research further, should you be interested.
Answer the questions below
What is the traditionally predominant server-side scripting language?
PHP
✓ Correct Answer
When validating by file extension, what would you call a list of accepted extensions (whereby the server rejects any extension not in the list)?
Whitelist
✓ Correct Answer
[Research] What MIME type would you expect to see when uploading a CSV file?
text/csv
✓ Correct Answer
Task 7 Bypassing Client-Side Filtering
Task 8 Bypassing Server-Side Filtering: File Extensions
Task 9 Bypassing Server-Side Filtering: Magic Numbers

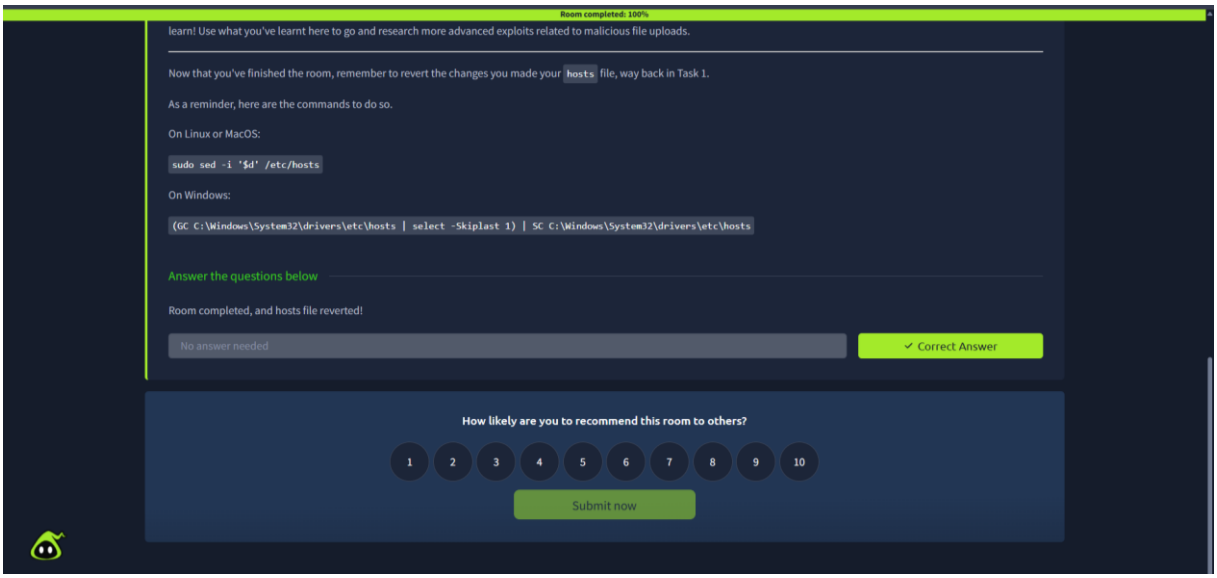

Room completed: 100%
root pts/2 tmux(1927).%1 15:47 3:18m 0.37s 0.07s vim ../../.
./overwrite/index.php
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ whoami
www-data
$
We've covered in detail two ways to bypass a Client-Side file upload filter. Now it's time for you to give it a shot for yourself! Navigate to java.uploadvulns.thm and bypass the filter to get a reverse shell. Remember that not all client-side scripts are inline! As mentioned previously, Gobuster would be a very good place to start here -- the upload directory name will be changing with every new challenge.
Answer the questions below
What is the flag in /var/www/?
THM{NDllZDQxNjJjOTE0YWNhZGY3YjljNmE2}
Correct Answer
Task 8 Bypassing Server-Side Filtering: File Extensions
Task 9 Bypassing Server-Side Filtering: Magic Numbers


Room completed: 100%
This is by no means an exhaustive list of upload vulnerabilities related to file extensions. As with everything in hacking, we are looking to exploit flaws in code that others have written; this code may very well be uniquely written for the task at hand. This is the really important point to take away from this task: there are a million different ways to implement the same feature when it comes to programming -- your exploitation must be tailored to the filter at hand. The key to bypassing any kind of server side filter is to enumerate and see what is allowed, as well as what is blocked; then try to craft a payload which can pass the criteria the filter is looking for.
Now your turn. You know the drill by now -- figure out and bypass the filter to upload and activate a shell. Your flag is in /var/www/. The site you're accessing is annex.uploadvulns.thm.
Be aware that this task has also implemented a randomised naming scheme for the first time. For now you shouldn't have any trouble finding your shell, but be aware that directories will not always be indexable...
Answer the questions below
What is the flag in /var/www/?
THM{MGEyYzJiYmI3ODIyM2FlNTNkNjZjYjFl}
Correct Answer
Task 9 Bypassing Server-Side Filtering: Magic Numbers
Task 10 Example Methodology
Task 11 Challenge [Optional]
Task 12 Conclusion


root pts/0 192.168.1.151 20:34 1:20m 0.03s 0.00s tmux a -t Dev
root pts/1 tmux(129038).%0 Mon15 1:20m 0.69s 4.23s tmux new -s Dev
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ whoami
www-data
There we have it -- we bypassed the server-side magic number filter and received a reverse shell.
Head to magic.uploadvulns.thm -- it's time for the last mini-challenge.
This will be the final example website you have to hack before the challenge in task eleven; as such, we are once again stepping up the level of basic security. The website in the last task implemented an altered naming scheme, prepending the date and time of upload to the file name. This task will not do so to keep it relatively easy; however, directory indexing has been turned off, so you will not be able to navigate to the directory containing the uploads. Instead you will need to access the shell directly using its URI.
Bypass the magic number filter to upload a shell. Find the location of the uploaded shell and activate it. Your flag is in /var/www/.
Answer the questions below
Grab the flag from /var/www/
THM{MWYSZGU4NzE0ZDlhNjE1NGM4ZThjZDJh}
✓ Correct Answer
Task 10 Example Methodology
Task 11 Challenge [Optional]


Room completed: 100%
would expect our upload to be stopped by a server side filter, but the error message that it gives us can be extremely useful in determining our next steps.
Assuming that our malicious file upload has been stopped by the server, here are some ways to ascertain what kind of server-side filter may be in place:
If you can successfully upload a file with a totally invalid file extension (e.g. testingimage.invalidfileextension) then the chances are that the server is using an extension blacklist to filter out executable files. If this upload fails then any extension filter will be operating on a whitelist.
Try re-uploading your originally accepted innocent file, but this time change the magic number of the file to be something that you would expect to be filtered. If the upload fails then you know that the server is using a magic number based filter.
As with the previous point, try to upload your innocent file, but intercept the request with Burpsuite and change the MIME type of the upload to something that you would expect to be filtered. If the upload fails then you know that the server is filtering based on MIME types.
Enumerating file length filters is a case of uploading a small file, then uploading progressively bigger files until you hit the filter. At that point you'll know what the acceptable limit is. If you're very lucky then the error message of original upload may outright tell you what the size limit is. Be aware that a small file length limit may prevent you from uploading the reverse shell we've been using so far.
You should now be well equipped to take on the challenge in task eleven.
Answer the questions below
Read the example methodology
No answer needed
✓ Correct Answer
Task 11 Challenge [Optional]
Task 12 Conclusion


Room completed: 100%
Task 10 Example Methodology
Task 11 Challenge [Optional]
Author's Note: After years of confusion, I have made the challenge optional. The following instructions still work, and the challenge is still available in the provided VM should you choose to attempt it.
Download Task Files
Be aware that the challenge is a big step up from the walkthrough content in the room, and contains a twist which you are very unlikely to see in the real world.
It's challenge time!
Head over to jewel.uploadvulns.thm.
Take what you've learned in this room and use it to get a shell on this machine. As per usual, your flag is in /var/www/. Bear in mind that this challenge will be an accumulation of everything you've learnt so far, so there may be multiple filters to bypass. The attached wordlist might help. Also remember that not all webservers have a PHP backend...
If you need a help, there are a series of hints here.
Additionally, there is a full video walkthrough available for this challenge here.
Answer the questions below
Hack the machine and grab the flag from /var/www/
No answer needed
✓ Correct Answer
Task 12 Conclusion



6. Impact

If exploitable, file upload vulnerabilities may lead to the following consequences:

- **Unauthorized file storage:** Unexpected or potentially harmful files may be stored on the server.
- **Sensitive information exposure:** Insecurely stored files may be accessible to unauthorized users.
- **Server-side code execution:** In certain configurations, uploaded executable files may be interpreted by the server.
- **Malware hosting:** An application may be abused to host harmful files.
- **Resource exhaustion:** Excessive or oversized uploads may consume storage, memory, or processing capacity.
- **Application integrity risks:** Unsafe file handling may affect application files or other stored content.
- **Business impact:** Successful exploitation may cause data exposure, service disruption, or reputational damage.

The actual severity depends on the confirmed weakness, the server's configuration, the permissions of the upload location, and the impact demonstrated during testing.

7. Remediation

The following measures help prevent file upload vulnerabilities.

1. Use an allowlist of permitted file types

Permit only the file formats required by the application. Do not rely exclusively on filename extensions.

2. Validate file content

Use appropriate server-side content checks and format-specific parsers. Treat client-supplied MIME types as untrusted input.

3. Generate safe server-side filenames

Use unpredictable or uniquely generated filenames instead of trusting user-supplied names. Prevent path manipulation and filename collisions.

4. Store uploads outside executable web directories

Where possible, store uploaded content in a separate storage location that does not execute files as server-side code.

5. Enforce file size limits

Apply reasonable maximum file sizes and request limits to reduce resource-exhaustion risks.

6. Apply authorization checks

Ensure that only authorized users can upload, view, modify, or delete files where the application's functionality requires access control.

7. Prevent unsafe file execution

Configure the server so that uploaded content cannot be interpreted as executable code.

8. Normalize and validate filenames

Reject invalid names and prevent path traversal or attempts to overwrite protected files.

9. Serve files securely

Use appropriate response headers and controlled download mechanisms when serving uploaded content. Consider isolating untrusted content on a separate origin where suitable.

10. Monitor and test upload functionality

Log relevant upload events, monitor suspicious activity, and regularly test validation, storage, authorization, and retrieval controls.

8. What You Learned

The main learning outcomes of this lab are:

- Understanding common web application file upload vulnerabilities.
- Recognizing the limitations of file extension and MIME-type checks.
- Understanding why server-side validation is necessary.
- Learning how insecure storage locations can increase risk.
- Understanding how server execution permissions affect upload security.
- Identifying the importance of safe file naming and storage.
- Learning to document upload tests with evidence and observed results.
- Understanding remediation techniques such as allowlists, content validation, least privilege, and non-executable storage.

Update these learning outcomes according to the tasks and techniques you actually completed in the room.

Conclusion

File upload functionality is a common attack surface in web applications. Weak validation, insecure storage, insufficient access controls, and unsafe server configuration can allow uploaded files to create significant security risks.

Secure implementation requires robust server-side validation, safe file naming, restricted storage locations, appropriate authorization, and controls that prevent uploaded content from being executed unexpectedly.

This report provides a structured approach to evaluating file upload security in an authorized lab environment. The final assessment should be based on actual observations, supporting screenshots, and the recorded completion status of the TryHackMe room.

Report Prepared By: Rohit Kushwah
Organization: IqraSec Academy
Internship: Web Application Penetration Testing (WAPT)